MagicCode Super-Resolution Software FAQ and Troubleshooting

Document Version: v1.2.0

1. General FAQ

What does MagicCode Super-Resolution do?

It increases image or frame resolution using MagicCode super-resolution algorithms with CPU or GPU acceleration paths. Public modes are Speed and Balanced.

Does the software upload my images or videos?

No. The technical reporting function does not upload image or video content. It may report technical fields such as device model, UUID, input dimensions, mode, scale, error code, report reason, and processing statistics as described in the Privacy Policy.

What scale factors are supported?

The public valid range is `1.0`x to `8.0`x. Actual support depends on backend and model configuration.

What are Speed and Balanced modes?

Speed prioritizes throughput and low latency. Balanced mode balances processing speed and output quality when more processing budget is available.

2. Initialization Problems

- **Model file cannot be opened:** verify `model_path`, packaging, runtime permissions, and app sandbox location.
- **Width or height out of range:** ensure input dimensions are between `64` and `4032`.
- **Scale out of range:** ensure `scaler_factor` is between `1.0` and `8.0`.
- **Backend unavailable:** confirm the selected backend is included in the build and supported by the device.
- **CPU/GPU input mismatch:** CPU backends require buffer input; GPU backends require texture input.

3. Processing Problems

- **Output is empty or all-zero:** confirm input pointer/texture validity, output allocation, backend context, and model match.
- **Unexpected output size:** query status and confirm scale and rounded output dimensions.
- **Processing fails after parameter change:** verify new model path, dimensions, scale, and mode are compatible.
- **Performance is lower than expected:** use release build, reduce input size, verify backend, reduce thermal throttling, and check thread count.

4. Unity Troubleshooting

- **DllNotFoundException on Android:** place `libmagic_sr_unity.so` under `Assets/Plugins/Android/arm64-v8a/`.
- **iOS symbol errors:** ensure `libmagic_sr_unity_ios.a` is linked into the Xcode target and required frameworks are present.
- **Model path errors:** use `StreamingAssets` or copy model files to a readable persistent path.
- **Texture processing failure:** verify input/output texture format matches the selected input type and backend.

5. Unreal Engine Troubleshooting

- **Plugin missing:** copy `MagicSR` into `<Project>/Plugins/MagicSR` and enable it.

- **Missing library:** verify platform `libmagic_sr.a` exists at the path expected by the module build file.
- **Android signature mismatch:** uninstall the existing package and reinstall.
- **Old UE Android build issue:** use an NDK version compatible with the selected UE version.
- **iOS packaging unavailable:** verify UE, Xcode, and Apple SDK compatibility.

6. Diagnostic Checklist

- Record MagicCode version from `MC_GetVersion`.
- Record backend, input type, mode, scale, width, height, and model path.
- Record API return values and `output_status_params_t.error_code`.
- Attach logs and minimal reproduction steps when contacting support.